

C Standards Roadmap: C89 → C99 → C11 → C17 → C23

Summary

C99 (ISO/IEC 9899:1999)

- ✓ Supported
- ✗ Skipped (Z80 limitations)
- ✓ Recently Added

C11 (ISO/IEC 9899:2011)

- ✓ Supported
- ✗ Skipped (Z80 limitations)
- ✓ Recently Added

C17 (ISO/IEC 9899:2018)

C23 (ISO/IEC 9899:2024)

- ✓ Supported
- ✗ Skipped (Z80 limitations)
- 📝 TODO

#embed — The Z80 Killer Feature

libc Headers Status

Predefined Macros

Implementation Priority

Phase 1 — Done ✓

Phase 2 — Next

Phase 3 — Future

Test Examples

Z80-Specific Considerations

C Standards Roadmap: C89 → C99 → C11 → C17 → C23

MinZ C frontend progression toward modern C standards on Z80.

Parser: modernc.org/cc/v4 — handles preprocessing, parsing, type checking. **Lowerer:** pkg/c89/lower.go — walks typed AST, emits HIR. **libc:** pkg/c89/libc/ — minimal headers for Z80 freestanding environment.

Summary

Standard	Features Added	Supported	Skipped (Z80)	TODO
C99	~15 major	12	2	1
C11	~10 major	4	4	2
C17	0 (bugfix)	✓ full	—	—
C23	~15 major	2	3	10

C99 (ISO/IEC 9899:1999)

✓ Supported

Feature	Implementation	Notes
// single-line comments	modernc.org/cc parser	
_Bool type	lower.go type mapping + boolNormalize()	Any non-zero → 1
for (int i = 0; ...)	IterationStatementForDecl case	
Mixed declarations and code	lowerCompound() processes interleaved decl/stmt	
Designated struct initializers { .x = 1 }	lowerStructInit()	Out-of-order, mixed positional
Compound literals (Type){...}	PostfixExpressionComplit case	Struct + scalar
inline functions	Parsed, treated as regular functions	No actual inlining at C level
restrict keyword	Parsed, ignored	Z80: no aliasing optimizer
<stdint.h>	libc/stdint.h	uint8_t through uint32_t
<stddef.h>	libc/stddef.h	

Feature	Implementation	Notes
		NULL, size_t, offsetof
<stdbool.h>	libc/stdbool.h	bool, true, false macros
Predefined bool/true/false	z80Predefined macros	C23-style, available without include

✗ Skipped (Z80 limitations)

Feature	Reason
Variable-length arrays (VLAs)	No runtime stack allocation on Z80. C11 made optional anyway.
_Complex / _Imaginary	No FPU on Z80.

✓ Recently Added

Feature	Implementation	Notes
Array designated init [i] = val	lowerArrayInit() in lower.go	Positional, designated, mixed, sparse — all work

C11 (ISO/IEC 9899:2011)

✓ Supported

Feature	Implementation	Notes
_Static_assert	modernc.org/cc parser	Compile-time assertions
_Generic	modernc.org/cc parser	Type-generic expressions
_Noreturn	Parsed, ignored	No codegen effect

Feature	Implementation	Notes
<code><stdalign.h></code>	<code>libc/stdalign.h</code>	Z80: always 1-byte aligned
<code><stdnoreturn.h></code>	<code>libc/stdnoreturn.h</code>	<code>noreturn</code> → <code>_Noreturn</code>
<code><assert.h></code>	<code>libc/assert.h</code>	<code>assert()</code> macro, <code>NDEBUG</code> support

✗ Skipped (Z80 limitations)

Feature	Reason
<code><threads.h></code>	Z80: single core, no OS threads
<code><stdatomic.h></code>	Z80: single core, all ops are atomic
<code>u8"/u"/U</code> string prefixes	Z80: ASCII only, no Unicode
<code>_Alignas(N)</code>	Z80: byte-addressed, alignment always 1

✓ Recently Added

Feature	Implementation	Notes
Anonymous structs/unions	<code>lowerStructDecl()</code> + <code>FieldOffset()</code>	Fields promoted into parent — no prefix. Uses cc parser's field offset for correct anonymous member resolution.
<code>_Alignof / _Alignof(expr)</code>	<code>lowerUnary()</code>	Returns 1 for all types (Z80 = byte-addressed)
<code>typeof</code>	Already worked via <code>moderncc.org/cc</code>	Parser resolves <code>typeof</code> to underlying type,

Feature	Implementation	Notes
		mapType() handles it

C17 (ISO/IEC 9899:2018)

C17 is a bugfix release — no new language features. Only clarifications and defect resolutions from C11.

✓ **Fully supported** by supporting C11.

`__STDC_VERSION__` is set to 201710L in `z80Predefined`.

C23 (ISO/IEC 9899:2024)

✓ **Supported**

Feature	Implementation	Notes
<code>bool/true/false</code> as keywords	<code>z80Predefined</code> macros	Available without <code>#include <stdbool.h></code>
<code>static_assert</code> (no message)	<code>assert.h</code> macro → <code>_Static_assert</code>	

✗ Skipped (Z80 limitations)

Feature	Reason
<code>_BitInt(N)</code>	Arbitrary-width integers — no Z80 support
<code>_Decimal32/64/128</code>	Decimal floating point — no FPU
<code>char8_t</code>	UTF-8 type — Z80 is ASCII

📋 TODO

Feature	Difficulty	Priority	Notes
<code>nullptr</code>	Small	Nice	

Feature	Difficulty	Priority	Notes
			Map to (void*)0. Check if cc.PrimaryExpressionNullptr exists.
typeof / typeof_unqual	Medium	Nice	Depends on modernc.org/cc support.
constexpr	Medium	Nice	Compile-time constant evaluation. Extend evalConstInit.
Enum underlying type enum E : uint8_t	Small	Nice	Useful for Z80 — saves bytes vs default int (2 bytes).
[[]] attributes	Small	Nice	Depends on parser. [[noreturn]], [[maybe_unused]].
Digit separators 1'000	Trivial	Nice	Depends on modernc.org/cc lexer.
#embed	Medium	High	Binary include — perfect for Z80 sprites, fonts, LUTs.
<stdbit.h>	Small	Nice	Bit manipulation: stdc_popcount, stdc_leading_zeros, etc.
<ctype.h> functions	✅ Done	—	libc/ctype.h — inline implementations, no lookup table.
auto type inference	Medium	Nice	Depends on parser. auto x = expr;

#embed — The Z80 Killer Feature

```
// C23: include binary data at compile time
const unsigned char font[] = {
    #embed "font.bin"    // 768 bytes of ZX Spectrum font data
};

const unsigned char sprite[] = {
    #embed "player.spr"  // sprite pixel data
};
```

```
const unsigned char sin_table[] = {
    #embed "sin256.bin" // precomputed sine lookup table
};
```

This replaces hand-typed DB sequences in assembly. High priority for Z80 development.

libc Headers Status

Header	Status	Contents
<stdint.h>	✓ Done	uint8_t..uint32_t, INT8_MIN..UINT32_MAX
<stddef.h>	✓ Done	NULL, size_t, ptrdiff_t, offsetof
<limits.h>	✓ Done	CHAR_BIT, INT_MAX, LONG_MAX, etc.
<stdbool.h>	✓ Done	bool, true, false, __bool_true_false_are_defined
<stdarg.h>	✓ Done	va_list, va_start, va_arg (basic)
<stdlib.h>	✓ Done	abs, atoi, malloc/free stubs
<string.h>	✓ Done	memcpy, strlen, strcmp, etc.
<math.h>	✓ Done	fabs, fmod stubs
<assert.h>	✓ Done	assert() macro + NDEBUG
<ctype.h>	✓ Done	Inline isdigit..toupper (no LUT)
<stdalign.h>	✓ Done	alignas, alignof (always 1 on Z80)
<stdnoreturn.h>	✓ Done	noreturn → _Noreturn
<stdio.h>	✗ N/A	Use @print metafunction instead
<errno.h>	📋 TODO	Error codes
<signal.h>	✗ N/A	No OS signals on Z80
<locale.h>	✗ N/A	No locale support
<time.h>	✗ N/A	No OS time on bare Z80
<stdbit.h>	📋 TODO	C23 bit manipulation

Predefined Macros

```
#define __Z80__ 1           /* Target architecture */
#define __MINZ__ 1         /* MinZ compiler */
#define __STDC__ 1         /* ISO C conformant */
#define __STDC_VERSION__ 201710L /* C17 conformance level */
#define __STDC_HOSTED__ 0  /* Freestanding (no OS) */
#define __SIZEOF_INT__ 2   /* 16-bit int */
#define __SIZEOF_POINTER__ 2 /* 16-bit pointers */
#define bool _Bool        /* C23 keyword */
#define true 1
#define false 0
```

Implementation Priority

Phase 1 — Done

1. `stdbool.h` — `bool/true/false`
2. `assert.h` — `assert()` macro
3. `ctype.h` — character classification (inline, no LUT)
4. `stdalign.h` — C11 alignment (trivial on Z80)
5. `stdnoreturn.h` — C11 `noreturn`
6. `Predefined macros` — `__STDC_VERSION__`, `bool/true/false`

Phase 2 — Next

1. Array designated initializers `[i] = val`
2. Anonymous structs/unions (C11)
3. `nullptr` (C23)

Phase 3 — Future

1. `#embed` (C23) — binary includes for sprites/fonts/LUTs
 2. `typeof` (C23)
 3. `constexpr` (C23)
 4. Enum underlying type (C23)
 5. `<stdbit.h>` (C23)
-

Test Examples

File	Features Tested
examples/c89/*.c	38 files — C89/C99 core features, 350 asserts
examples/c/ c99_stdbool.c	<stdbool.h>, _Bool, logical ops
examples/c/ c99_ctype.c	<ctype.h>, isdigit.toupper
examples/c/ c11_features.c	Designated init, compound literals, mixed decls, inline, for-init
examples/c/ c23_preview.c	bool without include, ternary chains
examples/c/ array_desig.c	Array designated init: positional, designated, mixed, sparse

Z80-Specific Considerations

- **int = 16 bits** — C integer promotion rules still apply, but `uint8_t` arithmetic stays 8-bit where safe (selective narrowing in lowerer)
- **No FPU** — float/double map to 4-byte stubs, no real arithmetic
- **No OS** — freestanding environment (`__STDC_HOSTED__ 0`), no stdio/signal/time
- **1-byte alignment** — `_Alignof(T) = 1` for all types
- **2-byte pointers** — 16-bit address space (64 KB)
- **long = 4 bytes** — but 32-bit arithmetic is expensive (multi-instruction sequences)

Last updated: 2026-03-26